

README.md

```
# Seed Computing
### A Unified Paradigm for Deterministic, Teleportable, and Self-Invariant
Computational Systems

[![DOI](https://zenodo.org/badge/DOI/10.5281/zenodo.XXXXXXX.svg)](https://doi
.org/10.5281/zenodo.XXXXXXX)
[![License: CC BY 4.0](https://img.shields.io/badge/License-CC%20BY%204.0-
lightgrey.svg)](https://creativecommons.org/licenses/by/4.0/)
[![Version](https://img.shields.io/badge/Version-1.0-blue.svg)]()
[![Filing Type](https://img.shields.io/badge/Filing-CIP%20Prior%20Art-
orange.svg)]()
```

```
**Inventor:** Leon Calvin Long II
**Publication Date:** July 18, 2026
**Document Type:** Technical Specification – Aurora Runtime Specification,
Version 1.0
**Filing Type:** Continuation-in-Part (CIP) – Prior Art Reference
**Field:** Theoretical Computing · Systems Architecture · Formal
Specification · Cryptographic Systems
```

> This repository is a public prior art disclosure in connection with a pending Continuation-in-Part (CIP) patent application by the inventor. All concepts, specifications, and technical disclosures herein are the original work of Leon Calvin Long II.

Abstract

Seed Computing is a theoretical framework for constructing complete computational systems – agents, state, behaviors, and emergent properties – deterministically from a single cryptographic origin value called an **AuroraSeed**. The Aurora Runtime Specification formalizes this paradigm across seven primary components: the Substrate Layer, Agent Mesh, Invariant Engine, Teleportation Engine, Cloaking Protocol, and the Unified Field Equation (UFE). The central claim is that an entire computational universe can be derived, reproduced, and relocated from a single 32-byte cryptographically signed seed value, with all state transitions governed by a pure, composable evolution operator.

Repository Contents

```
| File | Description |
|-----|-----|
| `whitepaper.pdf` | Full publication-ready prior art whitepaper |
| `README.md` | This file – spec summary and type definitions |
| `LICENSE` | Creative Commons Attribution 4.0 International |
```

The Three Axioms of Seed Computing

1. **Genesis Axiom** – All state is derivable from the seed S
2. **Locality Axiom** – Agents operate only on locally visible substrate
3. **Invariance Axiom** – Global invariants are preserved across all state transitions

Unified Field Equation (UFE)

...

$\Psi(t+1) = \Phi[\Psi(t), \Sigma(\text{agents}), \Lambda(\text{invariants}), T(\text{teleport_events}), K(\text{cloak_state})]$

...

Symbol	Meaning
$\Psi(t)$	Total universe state at tick t
Φ	Universal evolution operator (pure, deterministic)
Σ	Aggregate agent state vector
Λ	Invariant satisfaction tensor (1 = satisfied, 0 = violated)
T	Teleportation event set for tick t
K	Cloaking state map

Core Type Definitions

AuroraSeed

```typescript

```
AuroraSeed = {
 id : UUID, // Globally unique identifier for this seed
 entropy : Bytes[32], // 256-bit cryptographically secure entropy (SHA-256 aligned)
 timestamp : Epoch, // Unix epoch at seed generation (monotonic)
 signature : Ed25519 // Ed25519 signature over SHA-512(id || entropy || timestamp)
}
```

...

### ### SeedState

```typescript

```
SeedState = {
  universe_id : UUID,
  tick        : UInt64,
  entropy_pool : Bytes[],
  active_agents : Agent[],
  substrate    : Substrate
}
// Transition function:  $\delta(S_t) \rightarrow S_{t+1}$ 
```

...

Substrate & Cell

```typescript

```

Substrate = {
 topology : Graph<Node, Edge>,
 memory_map : Map<Address, Cell>,
 coherence_clock : UInt64
}

Cell = {
 value : Any,
 generation : UInt64,
 owner : AgentID | None
}
...

Agent & AgentState
```typescript
Agent = {
  id          : AgentID,
  seed_offset : UInt64,
  state       : AgentState,
  capabilities : Capability[],
  mesh_links  : AgentID[]
}

AgentState = {
  local_memory : Map<Key, Value>,
  position     : SubstrateAddress,
  tick_last_active : UInt64
}
...

### Message (Agent Mesh)
```typescript
Message = {
 from : AgentID,
 to : AgentID,
 payload : Bytes,
 tick : UInt64,
 nonce : UInt64
}
...

Invariant
```typescript
Invariant = {
  id          : InvariantID,
  predicate   : Fn(SeedState) → Bool,
  severity    : Hard | Soft,
  on_violation : Action
}

Action = Rollback | Halt | Compensate | Log
...

### TeleportRequest / TeleportResult
```typescript
TeleportRequest = {
 agent_id : AgentID,

```

```

 destination : SubstrateAddress,
 tick : UInt64,
 auth_token : Bytes
}

TeleportResult = {
 success : Bool,
 new_position : SubstrateAddress,
 latency_ticks : UInt64
}
...

CloakRequest / CloakKey
```typescript
CloakRequest = {
    target      : AgentID | SubstrateRegion,
    cloak_depth : UInt8,      // 0 = fully visible · 255 = cryptographically
undetectable
    duration_ticks : UInt64,
    key            : CloakKey
}

CloakKey = {
    seed_derived : Bool,
    key_material : Bytes[32]
}
...

---

## Invariant Catalog

| ID | Name | Category | Severity | On Violation |
|----|-----|-----|-----|-----|
| INV-001 | Mesh Connectivity | Structural | Hard | Halt |
| INV-002 | No Orphan Agents | Structural | Hard | Rollback |
| INV-003 | Bidirectional Link Symmetry | Structural | Soft | Compensate |
| INV-004 | Substrate Coherence | Temporal | Hard | Halt |
| INV-005 | Monotonic Tick Counter | Temporal | Hard | Halt |
| INV-006 | Monotonic Generation Counter | Temporal | Hard | Rollback |
| INV-007 | State Fidelity (Teleport) | Semantic | Hard | Rollback |
| INV-008 | No Agent Duplication | Semantic | Hard | Halt |
| INV-009 | Tick Continuity (Teleport) | Semantic | Hard | Halt |
| INV-010 | Signature Validity | Cryptographic | Hard | Halt |
| INV-011 | Entropy Uniqueness | Cryptographic | Hard | Halt |

---

## Teleportation Protocol

1. Agent issues `TeleportRequest`
2. Invariant Engine pre-validates destination: coherence, ownership, capacity
3. Agent state serialized to seed-derived snapshot
4. Agent de-instantiated at source
5. Agent re-instantiated at destination from snapshot
6. Mesh links re-established via mesh healing protocol
7. `TeleportResult` returned

```

Genesis Function

...

$\Omega(\text{seed}) \rightarrow \text{Universe}$

...

Property	Description
Surjectivity	Ω covers the entire reachable state space
Determinism	Identical seeds always produce identical universes
Irreversibility	Universe state cannot be used to recover the seed

Citation

...

Long II, Leon Calvin. "Seed Computing: A Unified Paradigm for Deterministic, Teleportable, and Self-Invariant Computational Systems."

Aurora Runtime Specification, Version 1.0. July 18, 2026.

DOI: 10.5281/zenodo.XXXXXXX

...

License

Licensed under [Creative Commons Attribution 4.0 International] (LICENSE).

© 2026 Leon Calvin Long II. All rights reserved under the terms of CC BY 4.0.

> ****Patent Notice:**** This disclosure constitutes prior art for a pending Continuation-in-Part (CIP) patent application by Leon Calvin Long II. Patent rights are expressly reserved and not granted by this license.

LICENSE

Creative Commons Attribution 4.0 International Public License

Copyright (c) 2026 Leon Calvin Long II

By exercising the Licensed Rights (defined below), You accept and agree to be bound by the terms and conditions of this Creative Commons Attribution 4.0 International Public License ("Public License").

[Full CC BY 4.0 legal text:

<https://creativecommons.org/licenses/by/4.0/legalcode>]

PATENT NOTICE

This license does not grant any rights under any patent held or applied for by Leon Calvin Long II, including but not limited to any pending Continuation-in-Part (CIP) patent application covering the Seed Computing

paradigm, the Aurora Runtime Specification, or any component thereof (AuroraSeed, Substrate Layer, Agent Mesh, Invariant Engine, Teleportation Engine, Cloaking Protocol, and Unified Field Equation). All patent rights are expressly reserved by Leon Calvin Long II.

ATtribution REQUIREMENT

Long II, Leon Calvin. "Seed Computing: A Unified Paradigm for Deterministic, Teleportable, and Self-Invariant Computational Systems." Aurora Runtime Specification, Version 1.0. July 18, 2026.
DOI: 10.5281/zenodo.XXXXXXX
License: CC BY 4.0 — <https://creativecommons.org/licenses/by/4.0/>

One thing to do after Zenodo upload: replace every XXXXXXXX in the README badge URLs and citation block with your actual Zenodo DOI number. Zenodo assigns it the moment you publish the record. Want me to update the whitepaper cover with the DOI too once you have it?